МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №3

по курсу «Программирование графических процессоров»

Классификация и кластеризация изображений на GPU

Выполнил: Л.А. Фокин

Группа: М8О-401Б-23

Преподаватель: А.Ю. Морозов

Москва, 2026

Содержание

1 Условие **2**
2 Программное и аппаратное обеспечение **2**
3 Метод решения **3**
4 Описание программы **4**
5 Результаты **7**
6 Выводы **9**

1 Условие

Цель работы: реализовать классификацию изображения на GPU с использованием константной памяти и одномерной сетки потоков, сравнить время вычислений на CPU и GPU.

Вариант 3. Метод минимального расстояния

Для каждого класса задаётся выборка пикселей. Необходимо вычислить средний цвет класса и отнести каждый пиксель к классу с ближайшим средним цветом в пространстве RGB. При равенстве расстояний выбирается класс с меньшим номером.

Входные данные состоят из путей к исходному и выходному файлам, числа классов n_c и описания каждого класса: числа пикселей выборки и их координат (x, y). Ограничения: $n_c \leq 32$, размер выборки класса $n_p \leq 2^{19}$, $wh \leq 4 \cdot 10^8$. В начале файла изображения записаны два четырёхбайтовых целых числа w, h, за ними построчно следуют пиксели RGBA по четыре байта. Программа сохраняет цветовые компоненты и записывает в альфа-канал номер класса, начиная с нуля.

2 Программное и аппаратное обеспечение

Аппаратное обеспечение

- **Графический процессор (GPU):** NVIDIA GeForce RTX 5070
  - Compute Capability: 12.0
  - Объем глобальной памяти (Global Memory): 12 792 МБ ($\approx$ 12 ГБ) GDDR7
  - Объем разделяемой памяти на блок: 49 152 байт
  - Объем константной памяти: 65 536 байт
  - Количество 32-битных регистров на блок: 65 536
  - Количество потоковых мультипроцессоров (SM): 48
  - Максимальное количество нитей на блок: 1024
  - Максимальный размер блока: $(1024, 1024, 64)$
  - Максимальный размер сетки: $(2147483647, 65535, 65535)$
- **Центральный процессор (CPU):** AMD Ryzen 5 8400F 6-Core Processor (6 физических ядер, 12 логических потоков, базовая тактовая частота 4.2 ГГц, Max Boost до 4.7 ГГц, кэш L3 16 МБ)
- **Оперативная память (RAM):** 16 ГБ DDR5
- **Жесткий диск (Накопитель):** NVMe Solid State Drive (SSD) емкостью 1 ТБ, файловая система ext4

Программное обеспечение

- **Операционная система (OS):** Ubuntu 24.04 LTS (x86_64 Linux kernel 6.6)
- **Среда разработки (IDE / Редактор):** Zed Editor / Visual Studio Code
- **Компилятор CUDA:** NVIDIA CUDA Compiler Driver (nvcc) v12.0 (V12.0.140)
- **Компилятор C++ хоста:** g++ (Ubuntu 13.3.0)

3 Метод решения

Для класса j вычисляется средний цвет $\boldsymbol{\mu}_j = (\mu_{jr}, \mu_{jg}, \mu_{jb})$ по заданным пикселям. Класс пикселя $\boldsymbol{p} = (r, g, b)$ определяется выражением

$$j^* = \arg\min_j \left[(r - \mu_{jr})^2 + (g - \mu_{jg})^2 + (b - \mu_{jb})^2\right].$$

Для выбора ближайшего класса достаточно сравнить квадраты расстояний. Если расстояния равны, выбирается класс с меньшим номером.

Чтобы исключить ошибку округления при равенстве дробных средних, программа сохраняет для каждого класса суммы каналов S_r, S_g, S_b и число пикселей n. Квадрат расстояния равен

$$D = \frac{(rn - S_r)^2 + (gn - S_g)^2 + (bn - S_b)^2}{n^2}.$$

Программа сравнивает дроби перекрёстным умножением. Для вычисления старшей части произведения в CUDA используется `__umul64hi`. Это позволяет точно сравнить расстояния при заданных ограничениях.

CPU вычисляет суммы выборок, после чего программа копирует их в константную память GPU. Одномерная сетка имеет фиксированное число потоков. Каждый поток обрабатывает пиксели с шагом, равным числу потоков в сетке. Обозначим через $N = wh$ число пикселей изображения, а через S общее число пикселей во всех выборках. Сложность алгоритма составляет $O(S + Nn_c)$ по времени и $O(N + n_c)$ по памяти.

Источники и материалы

1. NVIDIA Corporation. *CUDA C++ Programming Guide*. URL: `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

2. Морозов А. Ю. Материалы курса: лабораторная работа №3, лекции «Память CUDA» и «Классификация», архив бинарных данных из условий. URL: `https://drive.google.com/drive/folders/1XqbrmRt70LJOwScFF4dptp5WR2sQnO-8`.

4 Описание программы

В файле `main.cu` находятся функции чтения и записи изображения, код вычисления сумм на CPU и CUDA-функция `classify`. Она читает параметры классов из константной памяти, выбирает класс каждого пикселя и записывает его номер в альфа-канал. Программа выводит ошибки ввода и CUDA в `stderr`. Для замеров конфигурацию сетки можно задать аргументами командной строки.

Файл `cpu_ref.cpp` содержит однопоточную реализацию классификации. Конвертер `converter.py` переводит изображения в бинарный формат и обратно. Для иллюстрации в отчёте каждому номеру класса присвоен условный цвет.

Исходный код программы (`main.cu`)

```
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <cuda_runtime.h>

#define CSC(call)                                                              \
do {                                                                           \
    cudaError_t res = call;                                                    \
    if (res != cudaSuccess) {                                                  \
        std::fprintf(stderr, "ERROR: %s:%d: %s\n",                             \
                     __FILE__, __LINE__, cudaGetErrorString(res));             \
        std::exit(0);                                                          \
    }                                                                          \
} while (0)

struct Center { uint32_t r, g, b, count; };
__constant__ Center centers[32];

__global__ void classify(uchar4* pixels, size_t count, int classes) {
    size_t index = blockIdx.x * blockDim.x + threadIdx.x;
    size_t stride = blockDim.x * gridDim.x;
    for (size_t i = index; i < count; i += stride) {
        uchar4 pixel = pixels[i];
        uint64_t best = 0, best_denominator = 1;
        int label = 0;
        for (int j = 0; j < classes; ++j) {
            Center c = centers[j];
            int64_t dr = (int64_t)pixel.x * c.count - c.r;
            int64_t dg = (int64_t)pixel.y * c.count - c.g;
            int64_t db = (int64_t)pixel.z * c.count - c.b;
            uint64_t distance = dr * dr + dg * dg + db * db;
            uint64_t denominator = (uint64_t)c.count * c.count;
            uint64_t left_hi = __umul64hi(distance, best_denominator);
            uint64_t right_hi = __umul64hi(best, denominator);
            if (j == 0 || left_hi < right_hi ||
                (left_hi == right_hi && distance * best_denominator < best * denominator)) {
                best = distance;
                best_denominator = denominator;
                label = j;
            }
        }
        pixel.w = (unsigned char)label;
        pixels[i] = pixel;
    }
}

void read_image(const char* path, int& w, int& h, uchar4*& data) {
    FILE* fp = fopen(path, "rb");
    if (!fp) {
        std::fprintf(stderr, "ERROR: cannot open %s\n", path);
        std::exit(0);
    }
    if (fread(&w, sizeof(int), 1, fp) < 1 || fread(&h, sizeof(int), 1, fp) < 1) {
        std::fprintf(stderr, "ERROR: broken image file %s\n", path);
        std::exit(0);
    }
    if (w <= 0 || h <= 0 || (size_t)w * h > 400000000) {
```

```
        std::fprintf(stderr, "ERROR: invalid image dimensions\n");
        std::exit(0);
    }
    data = (uchar4*)std::malloc(sizeof(uchar4) * w * h);
    if (!data) {
        std::fprintf(stderr, "ERROR: cannot allocate image\n");
        std::exit(0);
    }
    if (fread(data, sizeof(uchar4), (size_t)w * h, fp) < (size_t)w * h) {
        std::fprintf(stderr, "ERROR: broken image file %s\n", path);
        std::exit(0);
    }
    fclose(fp);
}

void write_image(const char* path, int w, int h, uchar4* data) {
    FILE* fp = fopen(path, "wb");
    if (!fp) {
        std::fprintf(stderr, "ERROR: cannot open %s for writing\n", path);
        std::exit(0);
    }
    bool ok = fwrite(&w, sizeof(int), 1, fp) == 1 &&
              fwrite(&h, sizeof(int), 1, fp) == 1 &&
              fwrite(data, sizeof(uchar4), (size_t)w * h, fp) == (size_t)w * h;
    if (fclose(fp) != 0) ok = false;
    if (!ok) {
        std::fprintf(stderr, "ERROR: cannot write %s\n", path);
        std::exit(0);
    }
}

int main(int argc, char** argv) {
    char path_in[1024], path_out[1024];
    if (!fgets(path_in, sizeof(path_in), stdin) || !fgets(path_out, sizeof(path_out), stdin)
    ) {
        std::fprintf(stderr, "ERROR: expected input and output paths\n");
        return 0;
    }
    for (char* s : {path_in, path_out}) {
        for (int i = 0; s[i]; ++i) {
            if (s[i] == '\n' || s[i] == '\r') s[i] = '\0';
        }
    }
    int blocks = 256, threads = 256;
    if (argc != 1) {
        char extra;
        if (argc != 3 || std::sscanf(argv[1], "%d%c", &blocks, &extra) != 1 ||
            std::sscanf(argv[2], "%d%c", &threads, &extra) != 1 ||
            blocks < 1 || threads < 1) {
            std::fprintf(stderr, "ERROR: invalid launch configuration\n");
            return 0;
        }
    }
    int w, h;
    uchar4* img = nullptr;
    read_image(path_in, w, h, img);
    int classes;
    if (std::scanf("%d", &classes) != 1 || classes < 1 || classes > 32) {
        std::fprintf(stderr, "ERROR: invalid class count\n");
        return 0;
    }
    Center means[32];
    for (int j = 0; j < classes; ++j) {
        int n;
        if (std::scanf("%d", &n) != 1 || n < 1 || n > (1 << 19)) {
            std::fprintf(stderr, "ERROR: invalid sample count\n");
            return 0;
        }
        uint64_t r = 0, g = 0, b = 0;
        for (int i = 0; i < n; ++i) {
            int x, y;
            if (std::scanf("%d%d", &x, &y) != 2 || x < 0 || x >= w || y < 0 || y >= h) {
                std::fprintf(stderr, "ERROR: invalid sample coordinate\n");
                return 0;
            }
            uchar4 p = img[(size_t)y * w + x];
```

```
            r += p.x; g += p.y; b += p.z;
        }
        means[j] = {(uint32_t)r, (uint32_t)g, (uint32_t)b, (uint32_t)n};
    }
    size_t bytes = sizeof(uchar4) * (size_t)w * h;
    uchar4* dev = nullptr;
    CSC(cudaMalloc(&dev, bytes));
    CSC(cudaMemcpy(dev, img, bytes, cudaMemcpyHostToDevice));
    CSC(cudaMemcpyToSymbol(centers, means, sizeof(Center) * classes));
    cudaEvent_t start, stop;
    CSC(cudaEventCreate(&start));
    CSC(cudaEventCreate(&stop));
    CSC(cudaEventRecord(start));
    classify<<<blocks, threads>>>(dev, (size_t)w * h, classes);
    CSC(cudaGetLastError());
    CSC(cudaEventRecord(stop));
    CSC(cudaEventSynchronize(stop));
    float ms;
    CSC(cudaEventElapsedTime(&ms, start, stop));
    std::fprintf(stderr, "kernel_ms = %.6f\n", ms);
    CSC(cudaMemcpy(img, dev, bytes, cudaMemcpyDeviceToHost));
    write_image(path_out, w, h, img);
    CSC(cudaEventDestroy(start));
    CSC(cudaEventDestroy(stop));
    CSC(cudaFree(dev));
    std::free(img);
    return 0;
}
```

5 Результаты

На бинарных изображениях `04.data` и `09.data` проверены оба примера варианта 3 из методички. Номера классов совпали с ответами, каналы RGB сохранились. Дополнительно проверены случайные данные для 32 классов, обработка неверных координат и выбор меньшего номера при равных расстояниях, в том числе для дробных средних.

Для замеров использованы синтетические изображения из четырёх повторяющихся цветов. Их размеры: 500×200, 2500×2000 и 20000×20000 пикселей. В каждом случае заданы четыре класса с одним пикселем в выборке. Последнее изображение достигает ограничения $4 \cdot 10^8$ пикселей. CPU-версия собрана с `g++ -O3` и работает в одном потоке. В таблице указано время классификации на CPU и время выполнения CUDA-функции на GPU, измеренное с помощью CUDA Events. В обоих случаях замер не включает чтение файлов и вычисление сумм выборок; для GPU также не учитывается передача данных. Каждая конфигурация запускалась один раз. Время указано в миллисекундах.

Таблица 1: Время классификации на CPU и GPU

Пикселей	⟪⟪ блоки, нити ⟫⟫	CPU, мс	GPU, мс
100 000	$(1, 32)$	1.075	2.282
100 000	$(1, 256)$	1.075	0.555
100 000	$(1, 1024)$	1.075	0.148
100 000	$(32, 256)$	1.075	0.017
100 000	$(256, 256)$	1.075	0.014
100 000	$(1024, 256)$	1.075	0.014
100 000	$(1024, 1024)$	1.075	0.016
5 000 000	$(1, 32)$	53.041	91.813
5 000 000	$(1, 256)$	53.041	17.702
5 000 000	$(1, 1024)$	53.041	10.116
5 000 000	$(32, 256)$	53.041	0.428
5 000 000	$(256, 256)$	53.041	0.184
5 000 000	$(1024, 256)$	53.041	0.161
5 000 000	$(1024, 1024)$	53.041	0.166
400 000 000	$(256, 256)$	4396.900	19.003
400 000 000	$(1024, 256)$	4396.900	16.383

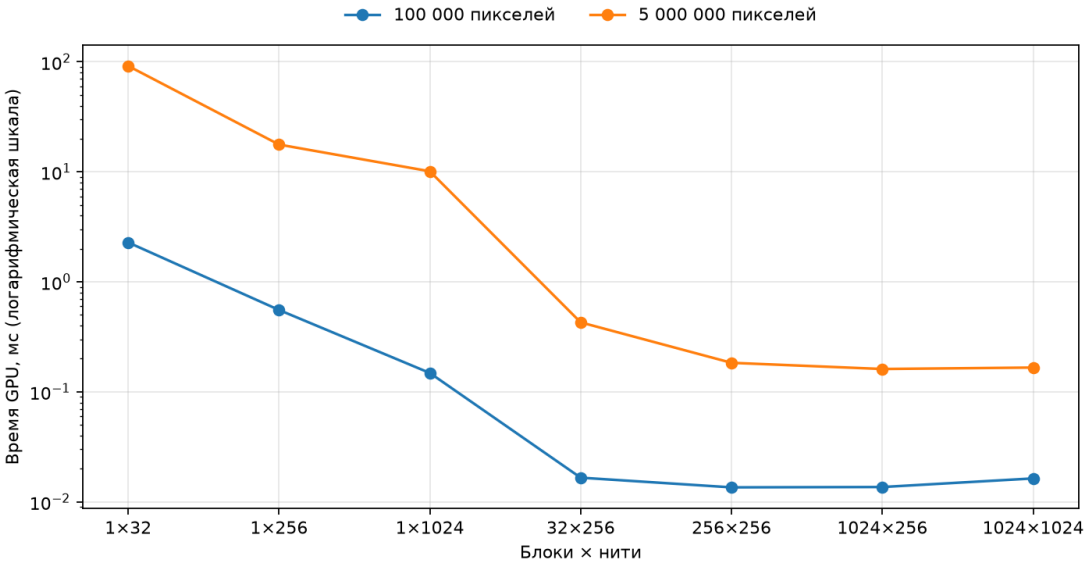


Рис. 1: Время GPU для разных конфигураций сетки

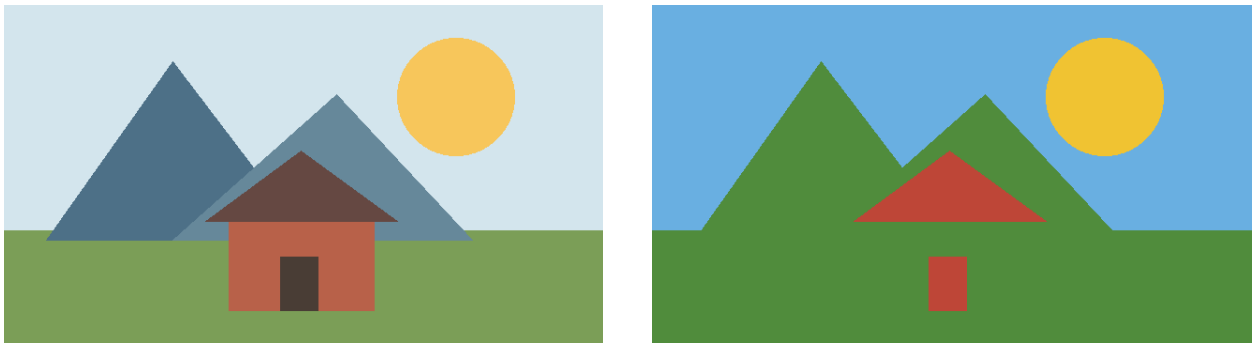


Рис. 2: Исходное синтетическое изображение и карта четырёх классов в условных цветах

6 Выводы

Метод минимального расстояния классифицирует пиксели по цвету. Его можно применять для выделения областей изображения, в том числе на спутниковых снимках, и для предварительной обработки изображений в задачах компьютерного зрения.

Основная трудность реализации заключалась в сравнении расстояний при дробных средних: округление могло нарушить правило выбора класса с меньшим номером. Проблему решило целочисленное сравнение дробей. Параметры классов хранятся в константной памяти; пиксели обрабатывает одномерная сетка потоков.

На изображении из 400 000 000 пикселей при конфигурации $(1024, 256)$ CPU затратил 4396,900 мс, GPU затратил 16,383 мс. На 5 000 000 пикселей время GPU при конфигурации $(1, 32)$ составило 91,813 мс, при $(1024, 1024)$ оно составило 0,166 мс. Во втором случае работу можно распределить между большим числом потоков. Влияние конфигурации зависит от размера изображения и накладных расходов запуска. Каждый результат получен за один запуск, а передача изображения на GPU не учитывается, поэтому определить полное ускорение программы по этим числам нельзя.